



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING
UTM Johor Bahru

DHL Knowledge Base System Project Report

Author: Ow Yee Hao (A23CS0261)

Course: SECJ3483 Web Technology & RPA
(DHL DAC 3.0 Challenge)

Scenario Selected: Scenario 1

Live Deployment: https://dhl_system_gio.vercel.app

Source Code: https://github.com/Owwwwyh/DHL_SYSTEM_GIO

Demo Video : <https://youtu.be/IJCuj5etHfI>

Date: 10 May 2026

1. Introduction & Scenario Description

In fast-paced logistics environments such as DHL warehouse operations, Standard Operating Procedures (SOPs) are continuously updated due to operational changes, safety requirements and process improvements. These updates are typically communicated through multiple informal and unstructured channels such as chat messages (Microsoft Teams, Telegram), long email threads, screenshots, handwritten notes and training materials.

While this information is essential for daily operations, it is highly fragmented and difficult to manage. As a result, organizations face several key challenges:

- Information is scattered across different platforms, making it difficult to establish a single source of truth
- Conflicting updates may occur without detection, leading to inconsistent or unsafe procedures
- There is no clear audit trail to track who created or modified SOPs

These issues significantly slow down SOP creation and reduce operational efficiency.

To address these problems, this project proposes an AI-powered Knowledge Base System that automates the transformation of unstructured data into structured knowledge articles. The system ingests raw inputs from both human users and an automated RPA bot, processes them using artificial intelligence and manages them through a controlled lifecycle consisting of Draft, Reviewed, Published and Archived states.

This approach ensures that all SOPs are standardized, traceable and easily accessible, providing a reliable and scalable solution for enterprise knowledge management.

2. System Architecture & Design

The system is designed based on a unified architecture where both human users and the RPA bot interact with the same backend API. This design eliminates duplication of business logic and ensures consistency across all operations.

The core system consists of the following components:

- Frontend and API layer implemented using Next.js 14 (App Router), allowing both user interface and backend logic to be deployed together
- PostgreSQL database hosted on Neon, managed using Prisma ORM for type-safe queries and structured schema management
- Google Gemini AI model used to extract structured content from unstructured inputs
- UiPath Studio used for RPA automation
- Vercel used for deployment and hosting

The overall data flow follows a pipeline approach. First, raw inputs are submitted either through the web interface or by the RPA bot using API requests. These inputs are then processed by the AI engine, which extracts structured information such as title, summary, procedural steps and tags. The processed data is stored in the database using a relational structure consisting of User, RawInput, Article and ArticleVersion entities. This ensures that every change is recorded and traceable.

A key design decision in this system is the use of a single REST API for both human users and the RPA bot. Instead of creating separate endpoints, both clients interact with the same API using different authentication methods (JWT for users and API key for the bot). This ensures a single source of truth and prevents inconsistencies between automated and manual processes.

3. Technical Explanation (API Data JSON)

The system uses a RESTful API to perform all data operations dynamically, ensuring that no data is hardcoded in the application. The API supports full CRUD functionality, including:

- Creation of raw inputs and articles through POST requests
- Retrieval of articles using GET requests with support for search, filtering and pagination
- Updating article content and workflow status using PUT and POST requests
- Soft deletion of articles using DELETE requests

All data is exchanged in JSON format and a consistent structure is enforced across the system. A typical article response includes fields such as title, summary, steps, tags and status. Arrays are used to represent structured lists such as procedural steps and tags, ensuring clarity and flexibility.

Example JSON structure:

```
None
{
  "id": "cmop8z2x40001abcdmnopqrst",
  "title": "Process Inbound RMA Parcels",
  "summary": "Step by step procedure for receiving and inspecting returned merchandise.",
  "steps": [
    "Step 1: Scan the RMA barcode.",
    "Step 2: Inspect the parcel for visible damage."
  ],
  "tags": ["rma", "returns", "inbound"],
  "status": "draft",
  "hasConflict": false,
  "createdAt": "2026-05-03T08:14:00Z"
}
```

To ensure system reliability and data integrity, several defensive mechanisms are implemented:

- Duplicate detection using file hashing to prevent repeated processing within a 14-day window
- Rate limiting to prevent excessive API usage
- Validation checks before database operations
- Fallback handling in case of AI processing failures

These mechanisms ensure that the system remains stable even under high load or unexpected errors.

4. RPA Workflow & Integration Explanation

The RPA component is developed using UiPath Studio and is responsible for automating the ingestion of external data into the system. The workflow is designed to operate in an unattended manner, allowing continuous processing without human intervention.

The automation process follows these steps:

- The bot scans a predefined source such as a Google Drive folder, email inbox, or local directory
- For each file, a hash is generated and sent to the system to check for duplicates
- If the file is new, it is uploaded to the system and processed using the AI engine
- The generated article is optionally promoted to a reviewed state
- After processing all files, a summary report is generated

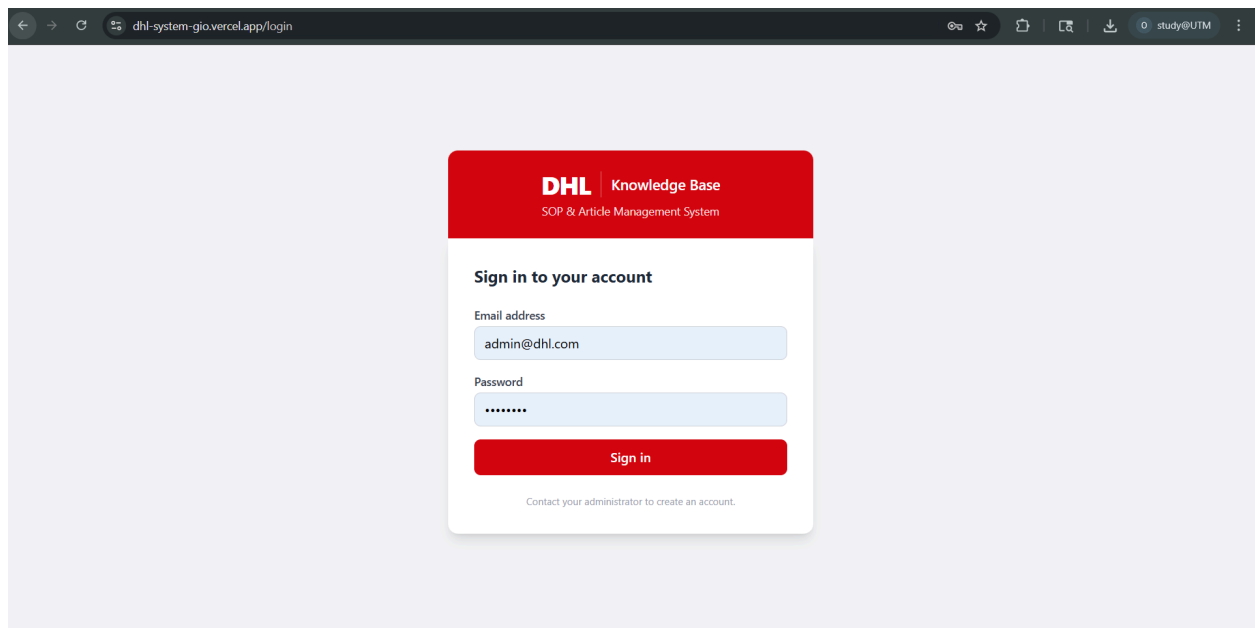
The workflow includes robust error handling mechanisms. Each file is processed within a Try/Catch block and failures trigger a retry mechanism with exponential backoff. If the error persists after multiple attempts, a screenshot is captured and logged and the process continues with the next file. This ensures that a single failure does not interrupt the entire workflow.

The RPA bot integrates directly with the web application using the same REST API. Authentication is handled using an API key, allowing secure communication without requiring user login sessions. This unified integration ensures consistent data handling and simplifies system maintenance.

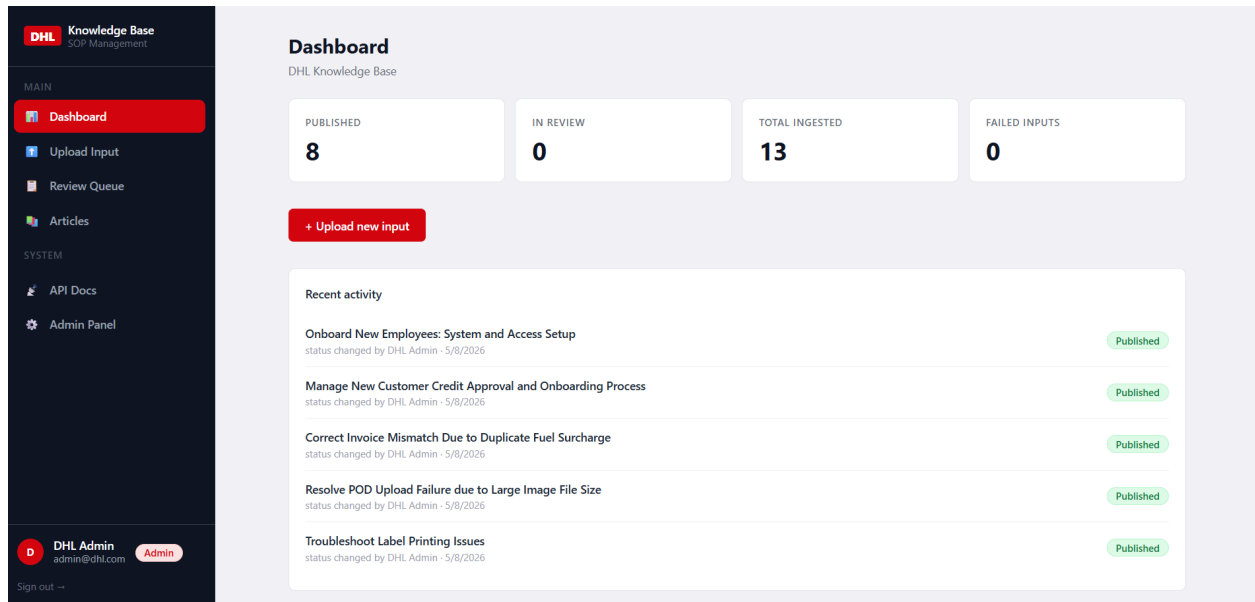
5. User Interface Design and Implementation

The user interface is designed to be simple, intuitive and efficient for warehouse staff and administrators. It provides all necessary functionality for managing SOPs without requiring technical expertise.

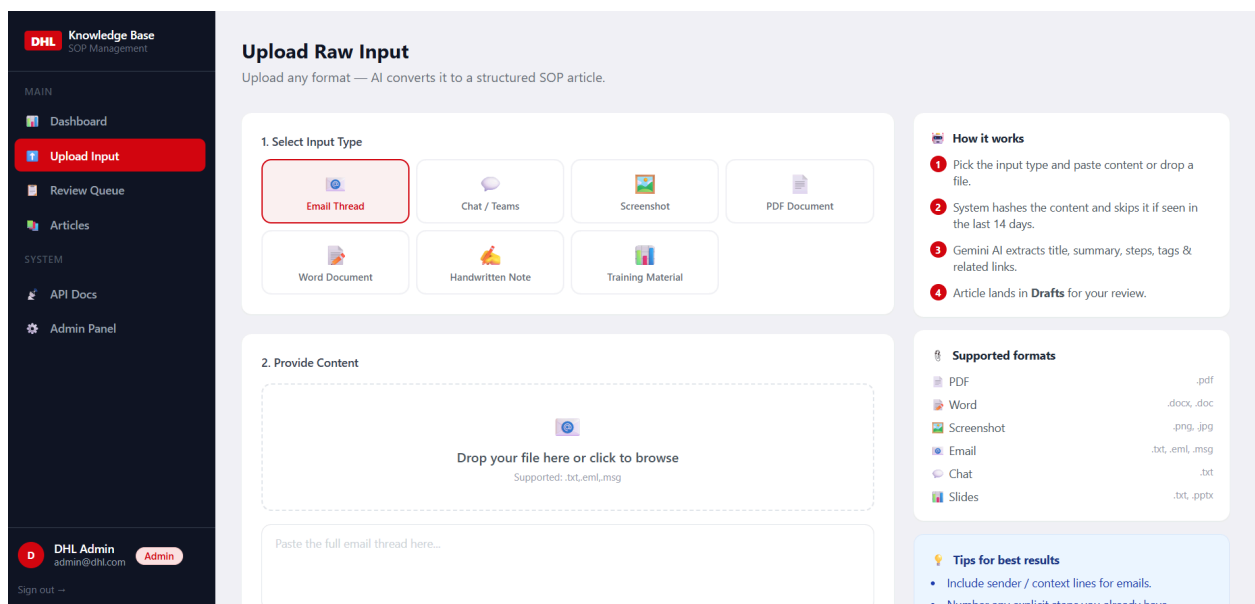
The system includes the following key interfaces:



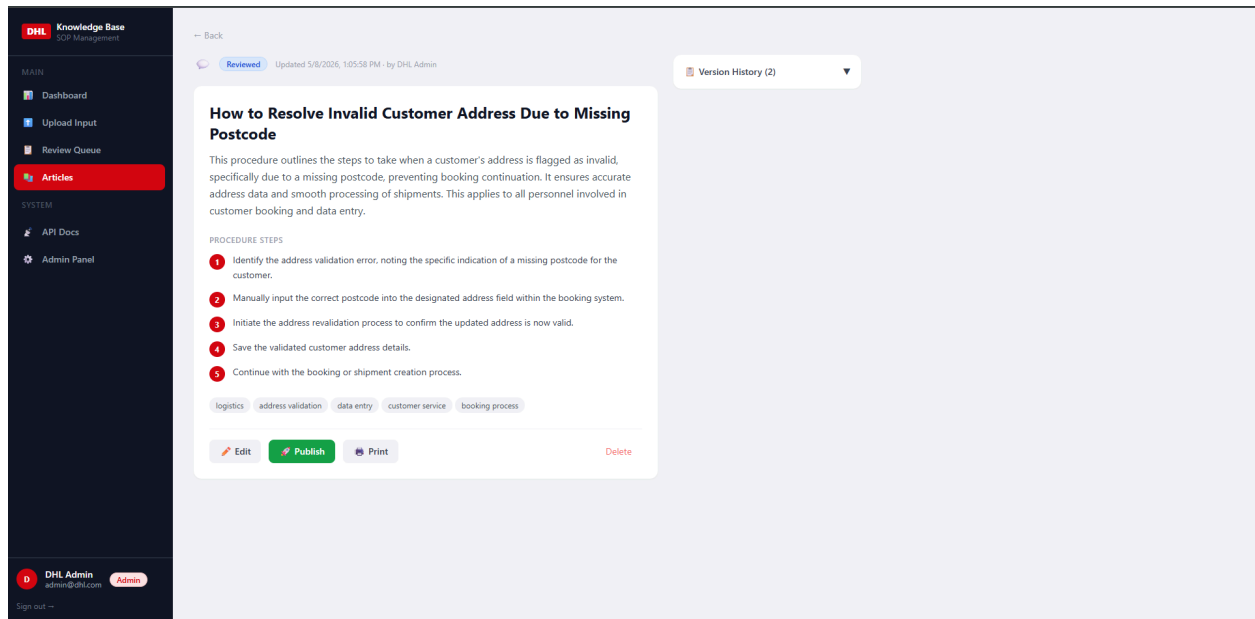
- A secure login page that enforces authentication and requires first-time users to change their password



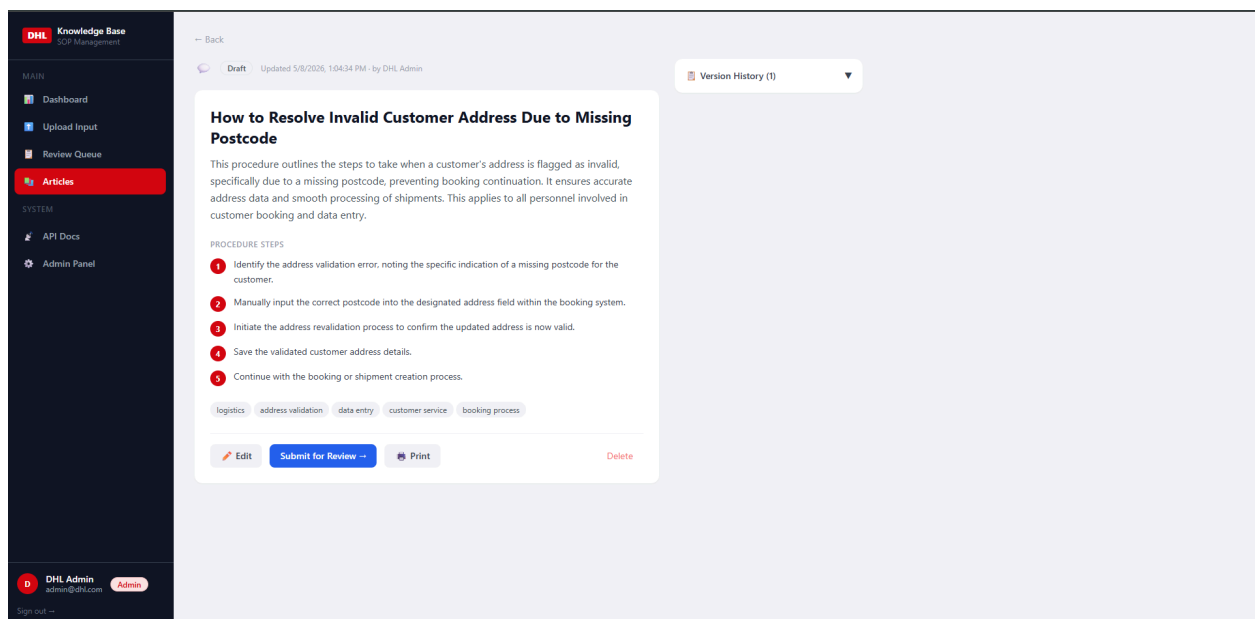
- A dashboard displaying key system statistics and recent activity



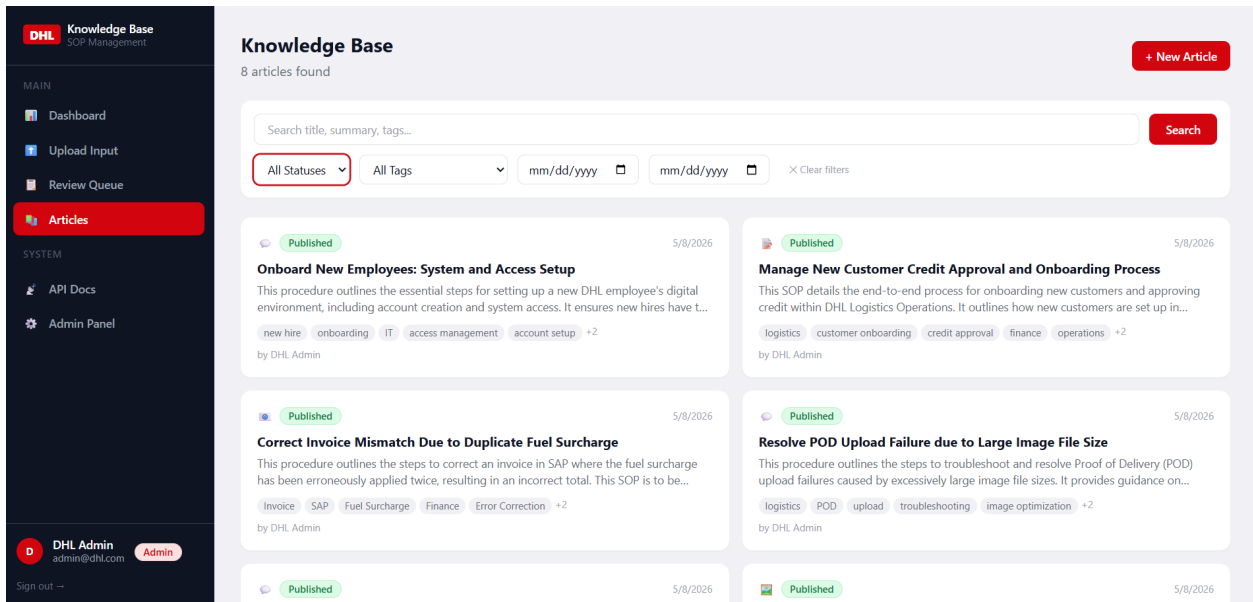
- An upload interface that supports multiple input formats, including text, PDF, DOCX and images, with drag-and-drop functionality



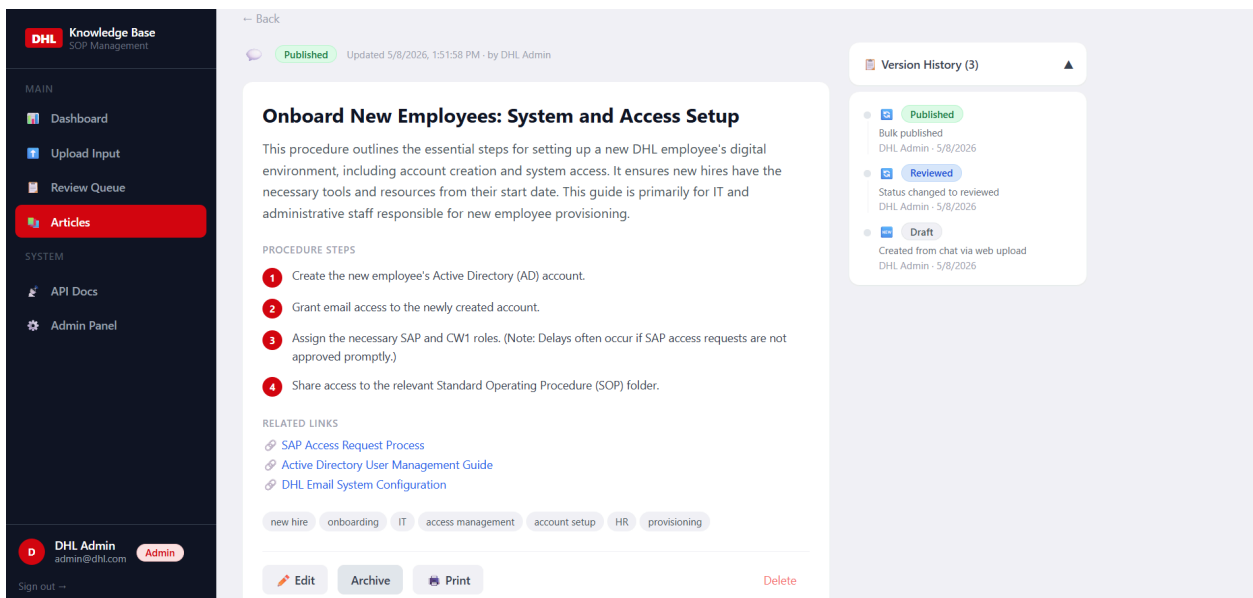
- Output showing the system transforming the user’s raw input into a well-structured and clean article using the Gemini 2.5 Flash LLM.



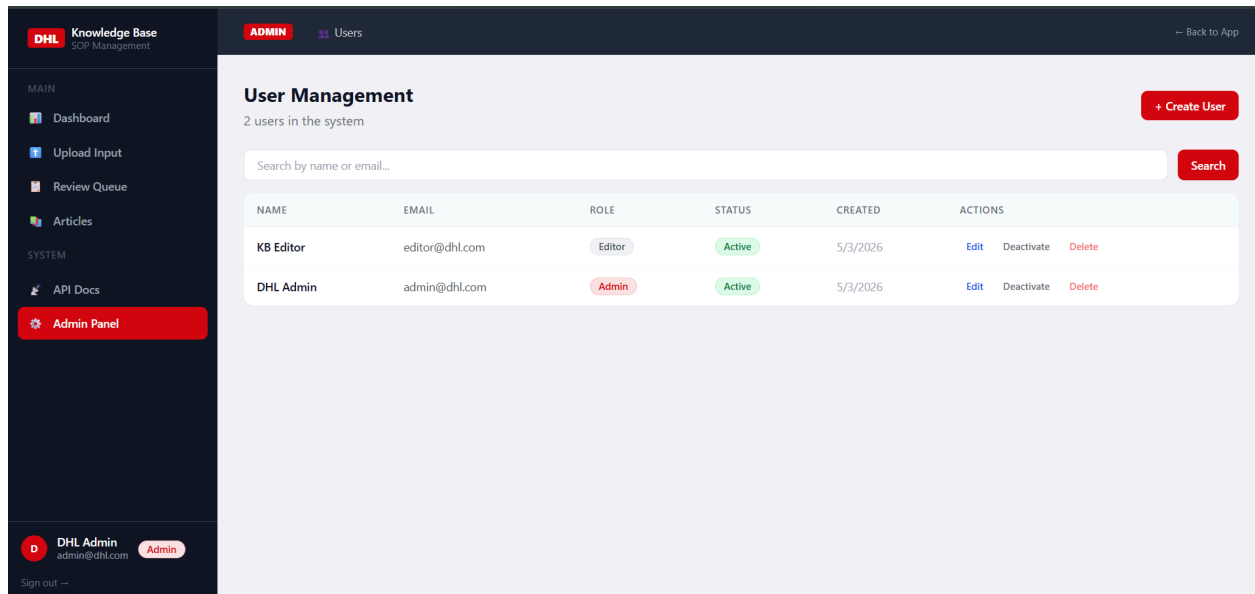
- Press “Submit for Review” to allow the admin to review and publish the article.



- A knowledge viewer that allows users to search and filter articles based on keywords, tags, dates and status



- An article detail page that displays structured SOP content along with version history and status information



- User Management Page that enables administrators to manage and control user accounts.

The interface is built using modern web technologies and provides responsive feedback to user actions, ensuring a smooth and user-friendly experience.

6. User Manual

To run the system locally, the following steps should be followed:

- Clone the project repository and install dependencies using npm
- Configure environment variables, including database connection, authentication secret, AI API key and RPA API key
- Initialize the database and seed default users
- Start the development server and access the application through a web browser

To execute the RPA workflow:

- Open the UiPath project in UiPath Studio
- Configure the required variables such as API base URL and API key
- Place sample input files in the designated folder
- Run the workflow to automatically process and upload the data

The system will handle ingestion, processing and logging automatically and a summary report will be generated upon completion.

7. Conclusion

The DHL Knowledge Base System provides an effective solution to the challenges of managing unstructured operational data in logistics environments. By combining web technologies, artificial intelligence and robotic process automation, the system is able to transform raw inputs into structured and standardized knowledge articles.

The use of a unified API ensures consistency between human and automated processes, while features such as version control, conflict detection and duplicate prevention enhance reliability and accuracy. The system is scalable, practical and aligned with real-world industry requirements, making it suitable for deployment in enterprise environments.